

Cours MongoDB Complet

Manipulation de données avec MongoDB

Guide pratique et complet

CRUD Requêtes Opérateurs Agrégation Performance

Ce que vous allez apprendre :

- Concepts de base MongoDB
- Opérations CRUD complètes
- Requêtes et filtres avancés
- Framework d'agrégation
- Optimisation et indexation
- Bonnes pratiques MongoDB

December 17, 2025

Contents

1	Les bases de MongoDB	3
1.1	Concepts clés	3
1.2	Connexion à MongoDB	3
1.3	Création (Sélection) de la base de données et de la collection)	3
2	Les opérations CRUD fondamentales	4
2.1	Insertion de documents (Create)	4
2.1.1	insertOne() Insérer un seul document	4
2.1.2	insertMany() Insérer plusieurs documents	4
2.2	Lecture de documents (Read)	5
2.2.1	find() Rechercher plusieurs documents	5
2.2.2	findOne() Rechercher un seul document	5
2.2.3	Projections Sélectionner les champs	5
2.2.4	Limiter et paginer les résultats	5
2.3	Mise à jour de documents (Update)	5
2.3.1	updateOne() Modifier un seul document	5
2.3.2	updateMany() Modifier plusieurs documents	6
2.3.3	Upsert Créer si inexistant	6
2.4	Suppression de documents (Delete)	7
2.4.1	deleteOne() Supprimer un seul document	7
2.4.2	deleteMany() Supprimer plusieurs documents	7
3	Les opérateurs de requête	7
3.1	Opérateurs de comparaison	7
3.1.1	Exemples pratiques	7
3.2	Opérateurs logiques	8
3.2.1	Exemples pratiques	8
3.3	Opérateurs d'élément	8
3.4	Opérateurs de tableau	9
3.5	Opérateurs d'évaluation	9
4	Les opérateurs de mise à jour	10
4.1	Opérateurs de champ	10
4.2	Opérateurs de tableau	11
5	Tri et organisation des résultats	11
5.1	Tri simple avec sort()	11
5.2	Tri multiple	12
6	Le framework d'agrégation	12
6.1	Structure de base	12
6.2	Les étapes principales	12
6.2.1	\$match Filtrer les documents	12
6.2.2	\$group Regrouper et agréger	13
6.2.3	\$project Remodeler les documents	13
6.2.4	\$sort Trier les résultats	13
6.2.5	\$unwind Décomposer les tableaux	14
6.2.6	\$lookup Jointures entre collections	14
6.2.7	\$facet Pipelines parallèles	14

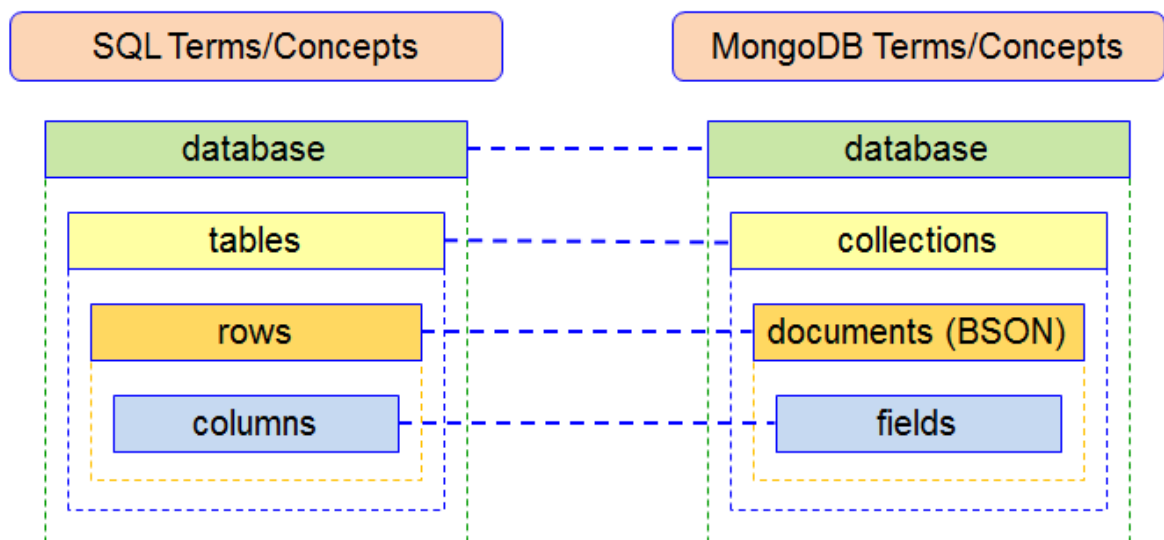
7	L'indexation pour la performance	15
7.1	Créer des index	15
7.2	La règle ESR	15
7.3	Analyser les performances	16
8	Bonnes pratiques	16
8.1	Conception du schéma	16
8.1.1	Quand imbriquer (embedding)	16
8.1.2	Quand référencer	16
8.2	Erreurs courantes à éviter	17
8.3	Optimisation des requêtes	17
9	Exemple pratique complet	17
9.1	Tableau de bord des stagiaires	17
10	Résumé des commandes essentielles	18
10.1	Index recommandés pour la collection Stagiaires	18

1 Les bases de MongoDB

MongoDB est une base de données NoSQL orientée documents, utilisant des structures de données flexibles basées sur JSON appelées BSON. Avant de plonger dans les opérations avancées, il est essentiel de comprendre les concepts fondamentaux.

1.1 Concepts clés

- **Base de données (Database)** : Conteneur logique pour les collections.
- **Collection** : Ensemble de documents similaires, équivalent à une table dans les bases relationnelles.
- **Document** : Enregistrement individuel au format BSON (Binary JSON), composé de paires clé-valeur.
- **Champ (Field)** : Attribut d'un document, similaire à une colonne dans une table relationnelle.
- **Index** : Structure de données qui améliore la vitesse des opérations de lecture.



1.2 Connexion à MongoDB

Pour interagir avec MongoDB, vous pouvez utiliser le shell MongoDB, des pilotes pour divers langages de programmation, ou des outils graphiques comme MongoDB Compass. Voici comment se connecter via le shell :

```
1 mongo "mongodb+srv://<cluster-url>" --username <username> --password <password>
```

1.3 Création (Sélection) de la base de données et de la collection)

Pour créer ou sélectionner une base de données, utilisez la commande suivante :

```
1 use <nom_de_la_base_de_données>
```

Pour créer ou sélectionner une collection, vous pouvez soit insérer un document dans une collection non-existante, soit utiliser la commande suivante :

```
1 db.createCollection("<nom_de_la_collection>")
```

2 Les opérations CRUD fondamentales

Les opérations CRUD (Create, Read, Update, Delete) constituent le cur de toute interaction avec MongoDB. Maîtriser ces quatre opérations vous permettra de manipuler efficacement n'importe quelle collection.

2.1 Insertion de documents (Create)

MongoDB offre deux méthodes principales pour insérer des données dans une collection.

2.1.1 insertOne() Insérer un seul document

```
1 db.Stagiaires.insertOne({
2   nom: "Dupont",
3   prenom: "Marie",
4   age: 24,
5   ville: "Paris",
6   competences: ["JavaScript", "Python"],
7   dateInscription: new Date("2024-09-01")
8 })
```

2.1.2 insertMany() Insérer plusieurs documents

```
1 db.Stagiaires.insertMany([
2   {
3     nom: "Martin",
4     prenom: "Lucas",
5     age: 28,
6     ville: "Lyon"
7   },
8   {
9     nom: "Bernard",
10    prenom: "Sophie",
11    age: 22,
12    ville: "Marseille"
13  }
14 ])
```

✓ Conseil

Points importants :

- Chaque document reçoit automatiquement un champ `_id` unique
- Les documents peuvent avoir des structures différentes (schéma flexible)
- Option `ordered`: `false` pour continuer en cas d'erreur

2.2 Lecture de documents (Read)

2.2.1 find() Rechercher plusieurs documents

```
1 // Tous les stagiaires
2 db.Stagiaires.find()
3
4 // Stagiaires de Paris
5 db.Stagiaires.find({ ville: "Paris" })
6
7 // Stagiaires de plus de 25 ans a Lyon
8 db.Stagiaires.find({
9     ville: "Lyon",
10    age: { $gt: 25 }
11 })
```

2.2.2 findOne() Rechercher un seul document

```
1 // Premier stagiaire trouve avec ce nom
2 db.Stagiaires.findOne({ nom: "Dupont" })
3
4 // Premier stagiaire de plus de 30 ans
5 db.Stagiaires.findOne({ age: { $gt: 30 } })
```

2.2.3 Projections Sélectionner les champs

```
1 // Retourner uniquement nom et prenom (sans _id)
2 db.Stagiaires.find(
3     {},
4     { nom: 1, prenom: 1, _id: 0 }
5 )
6
7 // Exclure le champ competences
8 db.Stagiaires.find({}, { competences: 0 })
```

2.2.4 Limiter et paginer les résultats

```
1 // Les 5 premiers stagiaires
2 db.Stagiaires.find().limit(5)
3
4 // Pagination : sauter 10, prendre 5
5 db.Stagiaires.find().skip(10).limit(5)
6
7 // Compter les documents
8 db.Stagiaires.countDocuments({ ville: "Paris" })
```

2.3 Mise à jour de documents (Update)

2.3.1 updateOne() Modifier un seul document

```
1 // Changer l'age
2 db.Stagiaires.updateOne(
3   { nom: "Dupont", prenom: "Marie" },
4   { $set: { age: 25 } }
5 )
6
7 // Ajouter un champ email
8 db.Stagiaires.updateOne(
9   { nom: "Martin" },
10  { $set: { email: "lucas.martin@email.com" } }
11 )
```

2.3.2 updateMany() Modifier plusieurs documents

```
1 // Ajouter le statut "actif" a tous les Parisiens
2 db.Stagiaires.updateMany(
3   { ville: "Paris" },
4   { $set: { statut: "actif" } }
5 )
6
7 // Incrementer l'age de tous de 1 an
8 db.Stagiaires.updateMany(
9   {},
10  { $inc: { age: 1 } }
11 )
```

! Attention

Erreur commune : Oublier \$set remplace TOUT le document !

```
1 // DANGER : Remplace tout le document
2 db.Stagiaires.updateOne(
3   { nom: "Dupont" },
4   { age: 26 } // Perd nom, prenom, ville...
5 )
6
7 // CORRECT : Met a jour seulement age
8 db.Stagiaires.updateOne(
9   { nom: "Dupont" },
10  { $set: { age: 26 } }
11 )
```

2.3.3 Upsert Créer si inexistant

```
1 // Met a jour si existe, cree sinon
2 db.Stagiaires.updateOne(
3   { email: "nouveau@email.com" },
4   { $set: {
5     nom: "Nouveau",
6     prenom: "Stagiaire"
7   } },
8   { upsert: true }
9 )
```

2.4 Suppression de documents (Delete)

2.4.1 deleteOne() Supprimer un seul document

```
1 db.Stagiaires.deleteOne({ nom: "Dupont" })
```

2.4.2 deleteMany() Supprimer plusieurs documents

```
1 // Supprimer tous les stagiaires de moins de 20 ans
2 db.Stagiaires.deleteMany({ age: { $lt: 20 } })
3
4 // Supprimer TOUS les documents
5 db.Stagiaires.deleteMany({})
```

3 Les opérateurs de requête

Les opérateurs permettent de construire des requêtes sophistiquées pour filtrer précisément vos données.

3.1 Opérateurs de comparaison

Opérateur	Description	Exemple
\$eq	Égal à	{ age: { \$eq: 25 } }
\$ne	Différent de	{ ville: { \$ne: "Paris" } }
\$gt	Supérieur à	{ age: { \$gt: 25 } }
\$gte	Supérieur ou égal	{ age: { \$gte: 18 } }
\$lt	Inférieur à	{ age: { \$lt: 30 } }
\$lte	Inférieur ou égal	{ age: { \$lte: 65 } }
\$in	Dans une liste	{ ville: { \$in: ["Paris", "Lyon"] } }
\$nin	Pas dans une liste	{ ville: { \$nin: ["Paris"] } }

Table 1: Opérateurs de comparaison MongoDB

3.1.1 Exemples pratiques

```
1 // Stagiaires entre 20 et 30 ans
2 db.Stagiaires.find({
3   age: { $gte: 20, $lte: 30 }
4 })
5
6 // Stagiaires de Paris, Lyon ou Marseille
7 db.Stagiaires.find({
8   ville: { $in: ["Paris", "Lyon", "Marseille"] }
9 })
10
11 // Stagiaires qui ne sont PAS de Paris
12 db.Stagiaires.find({
13   ville: { $nin: ["Paris"] }
```



```
14 })
```

3.2 Opérateurs logiques

Opérateur	Description
\$and	ET logique (toutes les conditions vraies)
\$or	OU logique (au moins une condition vraie)
\$not	Négation d'une condition
\$nor	NI l'un NI l'autre (toutes fausses)

Table 2: Opérateurs logiques

3.2.1 Exemples pratiques

```
1 // $and : Stagiaires de Paris ET de plus de 25 ans
2 db.Stagiaires.find({
3   $and: [
4     { ville: "Paris" },
5     { age: { $gt: 25 } }
6   ]
7 })
8
9 // Syntaxe implicite du $and (equivalent)
10 db.Stagiaires.find({
11   ville: "Paris",
12   age: { $gt: 25 }
13 })
14
15 // $or : Stagiaires de Paris OU de moins de 22 ans
16 db.Stagiaires.find({
17   $or: [
18     { ville: "Paris" },
19     { age: { $lt: 22 } }
20   ]
21 })
22
23 // $nor : Ni de Paris, ni de moins de 20 ans
24 db.Stagiaires.find({
25   $nor: [
26     { ville: "Paris" },
27     { age: { $lt: 20 } }
28   ]
29 })
```

3.3 Opérateurs d'élément

Opérateur	Description	Exemple
\$exists	Vérifie si un champ existe	{ email: { \$exists: true } }
\$type	Vérifie le type BSON	{ age: { \$type: "int" } }

Table 3: Opérateurs d'élément

```

1 // Stagiaires qui ont un email
2 db.Stagiaires.find({ email: { $exists: true } })
3
4 // Stagiaires sans numero de telephone
5 db.Stagiaires.find({ telephone: { $exists: false } })
6
7 // Stagiaires dont l'age est un entier
8 db.Stagiaires.find({ age: { $type: "int" } })

```

3.4 Opérateurs de tableau

Opérateur	Description
\$all	Contient TOUS les éléments spécifiés
\$elemMatch	Au moins un élément satisfait toutes les conditions
\$size	Taille exacte du tableau

Table 4: Opérateurs de tableau

```

1 // Stagiaires maitrisant JavaScript ET Python
2 db.Stagiaires.find({
3   competences: { $all: ["JavaScript", "Python"] }
4 })
5
6 // Stagiaires avec exactement 2 competences
7 db.Stagiaires.find({
8   competences: { $size: 2 }
9 })
10
11 // $elemMatch avec documents imbriqués
12 db.Stagiaires.find({
13   notes: {
14     $elemMatch: {
15       matiere: "MongoDB",
16       score: { $gt: 80 }
17     }
18   }
19 })

```

3.5 Opérateurs d'évaluation

Opérateur	Description
\$regex	Recherche par expression régulière
\$expr	Utilise des expressions d'agrégation
\$mod	Opération modulo

Table 5: Opérateurs d'évaluation

```

1 // Noms commençant par "Du" (insensible à la casse)
2 db.Stagiaires.find({
3   nom: { $regex: /^Du/i }
4 })

```

```

5
6 // Prenoms se terminant par "ie"
7 db.Stagiaires.find({
8   prenom: { $regex: /ie$/i }
9 })
10
11 // $mod : Stagiaires avec age pair
12 db.Stagiaires.find({
13   age: { $mod: [2, 0] }
14 })

```

4 Les opérateurs de mise à jour

4.1 Opérateurs de champ

Opérateur	Description
\$set	Définit la valeur d'un champ
\$unset	Supprime un champ
\$inc	Incrémente une valeur
\$mul	Multiplie une valeur
\$min	Met à jour si nouvelle valeur inférieure
\$max	Met à jour si nouvelle valeur supérieure
\$rename	Renomme un champ
\$currentDate	Définit la date actuelle

Table 6: Opérateurs de champ

```

1 // $set : Modifier plusieurs champs
2 db.Stagiaires.updateOne(
3   { nom: "Dupont" },
4   { $set: {
5     age: 26,
6     ville: "Bordeaux",
7     "adresse.codePostal": "33000"
8   }}
9 )
10
11 // $unset : Supprimer le champ telephone
12 db.Stagiaires.updateOne(
13   { nom: "Martin" },
14   { $unset: { telephone: "" } }
15 )
16
17 // $inc : Augmenter l'age de 1
18 db.Stagiaires.updateOne(
19   { nom: "Bernard" },
20   { $inc: { age: 1, credits: -10 } }
21 )
22
23 // $currentDate : Enregistrer date de modification
24 db.Stagiaires.updateOne(
25   { nom: "Dupont" },
26   {
27     $set: { statut: "valide" },
28     $currentDate: { derniereModification: true }

```

```

29   }
30 )

```

4.2 Opérateurs de tableau

Opérateur	Description
\$push	Ajoute un élément au tableau
\$pull	Retire les éléments correspondant à une condition
\$addToSet	Ajoute seulement si n'existe pas
\$pop	Retire le premier (-1) ou dernier (1) élément
\$pullAll	Retire toutes les valeurs spécifiées

Table 7: Opérateurs de tableau

```

1 // $push : Ajouter une competence
2 db.Stagiaires.updateOne(
3   { nom: "Dupont" },
4   { $push: { competences: "Docker" } }
5 )
6
7 // $push avec $each : Ajouter plusieurs competences
8 db.Stagiaires.updateOne(
9   { nom: "Martin" },
10  { $push: {
11      competences: {
12        $each: ["Kubernetes", "AWS", "Terraform"]
13      }
14    }}
15 )
16
17 // $addToSet : Ajouter seulement si pas deja present
18 db.Stagiaires.updateOne(
19   { nom: "Petit" },
20   { $addToSet: { competences: "Git" } }
21 )
22
23 // $pull : Retirer une competence specifique
24 db.Stagiaires.updateOne(
25   { nom: "Dupont" },
26   { $pull: { competences: "HTML" } }
27 )

```

5 Tri et organisation des résultats

5.1 Tri simple avec sort()

```

1 // Tri par age croissant (1 = ascendant)
2 db.Stagiaires.find().sort({ age: 1 })
3
4 // Tri par age decroissant (-1 = descendant)
5 db.Stagiaires.find().sort({ age: -1 })
6

```

```
7 // Tri alphabetique par nom
8 db.Stagiaires.find().sort({ nom: 1 })
```

5.2 Tri multiple

```
1 // Tri par ville (A-Z), puis par age (decroissant)
2 db.Stagiaires.find().sort({ ville: 1, age: -1 })
3
4 // Combiner tri, limite et projection
5 db.Stagiaires.find(
6     { ville: "Paris" },
7     { nom: 1, prenom: 1, age: 1, _id: 0 }
8 ).sort({ age: -1 }).limit(10)
```

✓ Conseil

Ordre des opérations : MongoDB applique automatiquement les opérations dans l'ordre optimal : filtre tri skip limit

6 Le framework d'agrégation

Le framework d'agrégation est l'outil le plus puissant de MongoDB pour transformer et analyser les données. Il fonctionne comme un **pipeline** où chaque étape transforme les documents.

6.1 Structure de base

```
1 db.collection.aggregate([
2     { $etape1: { ... } },
3     { $etape2: { ... } },
4     { $etape3: { ... } }
5 ])
```

6.2 Les étapes principales

6.2.1 \$match Filtrer les documents

Placez \$match au début pour filtrer tôt et bénéficier des index.

```
1 // Filtrer les stagiaires de Paris
2 db.Stagiaires.aggregate([
3     { $match: { ville: "Paris" } }
4 ])
5
6 // Filtrer par plusieurs criteres
7 db.Stagiaires.aggregate([
8     { $match: {
9         ville: { $in: ["Paris", "Lyon"] },
10        age: { $gte: 20, $lte: 30 }
11    }}
12 ])
```

6.2.2 \$group Regrouper et agréger

```

1 // Nombre de stagiaires par ville
2 db.Stagiaires.aggregate([
3   { $group: {
4     _id: "$ville",
5     nombreStagiaires: { $sum: 1 }
6   }}
7 ])
8
9 // Statistiques complètes par ville
10 db.Stagiaires.aggregate([
11   { $group: {
12     _id: "$ville",
13     total: { $sum: 1 },
14     ageMoyen: { $avg: "$age" },
15     ageMin: { $min: "$age" },
16     ageMax: { $max: "$age" },
17     stagiaires: { $push: "$nom" }
18   }}
19 ])

```

Accumulateur	Description
\$sum	Somme des valeurs
\$avg	Moyenne
\$min / \$max	Valeur minimum/maximum
\$first / \$last	Premier/dernier élément
\$push	Crée un tableau avec tous les éléments
\$addToSet	Tableau sans doublons

Table 8: Accumulateurs dans \$group

6.2.3 \$project Remodeler les documents

```

1 // Créer des champs calculés
2 db.Stagiaires.aggregate([
3   { $project: {
4     _id: 0,
5     nomComplet: {
6       $concat: ["$prenom", " ", "$nom"]
7     },
8     anneeNaissance: {
9       $subtract: [2025, "$age"]
10    },
11    estSenior: { $gte: ["$age", 30] },
12    nombreCompetences: {
13      $size: "$competences"
14    }
15  }}
16 ])

```

6.2.4 \$sort Trier les résultats

```
1 // Villes avec le plus de stagiaires d'abord
2 db.Stagiaires.aggregate([
3   { $group: {
4     _id: "$ville",
5     total: { $sum: 1 }
6   }},
7   { $sort: { total: -1 } }
8 ])
```

6.2.5 \$unwind Décomposer les tableaux

```
1 // Analyser les competences individuellement
2 db.Stagiaires.aggregate([
3   { $unwind: "$competences" },
4   { $group: {
5     _id: "$competences",
6     nombreStagiaires: { $sum: 1 }
7   }},
8   { $sort: { nombreStagiaires: -1 } }
9 ])
```

6.2.6 \$lookup Jointures entre collections

```
1 // Joindre les stagiaires avec leurs formations
2 db.Stagiaires.aggregate([
3   { $lookup: {
4     from: "Formations",
5     localField: "formationId",
6     foreignField: "_id",
7     as: "detailFormation"
8   }}
9 ])
```

6.2.7 \$facet Pipelines parallèles

```
1 // Tableau de bord complet en une requete
2 db.Stagiaires.aggregate([
3   { $facet: {
4     "parVille": [
5       { $group: {
6         _id: "$ville",
7         total: { $sum: 1 }
8       }},
9       { $sort: { total: -1 } }
10    ],
11     "statistiquesAge": [
12       { $group: {
13         _id: null,
14         moyenne: { $avg: "$age" }
15       }}
16    ]
17   }}
18 ])
```

```
18 | })
```

7 L'indexation pour la performance

Les index sont **essentiels** pour les performances. Sans index, MongoDB effectue un scan complet de la collection.

7.1 Créer des index

```
1 // Index simple
2 db.Stagiaires.createIndex({ nom: 1 })
3
4 // Index compose
5 db.Stagiaires.createIndex({ ville: 1, age: -1 })
6
7 // Index unique
8 db.Stagiaires.createIndex(
9     { email: 1 },
10    { unique: true }
11 )
12
13 // Index partiel
14 db.Stagiaires.createIndex(
15     { dateInscription: 1 },
16     {
17         partialFilterExpression: {
18             statut: "actif"
19         }
20     }
21 )
```

7.2 La règle ESR

Règle ESR pour les index composés :

1. Equality Champs d'égalité exacte
2. Sort Champs de tri
3. Range Champs de plage

```
1 // Pour cette requete :
2 db.Stagiaires.find({
3     ville: "Paris", // Equality
4     age: { $gt: 20 } // Range
5 }).sort({ nom: 1 }) // Sort
6
7 // L'index optimal est :
8 db.Stagiaires.createIndex({
9     ville: 1, // E
10    nom: 1, // S
11    age: 1 // R
12 })
```


7.3 Analyser les performances

```
1 // Analyser une requete
2 db.Stagiaires.find({ ville: "Paris" })
3   .explain("executionStats")
```

✓ Conseil

Métriques clés à surveiller :

- winningPlan.stage: "IXSCAN" → OK Utilise un index
- winningPlan.stage: "COLLSCAN" → NON Scan complet
- Ratio totalDocsExamined / nReturned proche de 1:1

8 Bonnes pratiques

8.1 Conception du schéma

Principe fondamental : Les données accédées ensemble doivent être stockées ensemble.

8.1.1 Quand imbriquer (embedding)

- Relation "contient" (un-à-un ou un-à-peu)
- Données toujours lues ensemble
- Pas de croissance illimitée
- Besoin d'opérations atomiques

```
1 // Bon : Adresses imbriquées
2 {
3   nom: "Dupont",
4   adresses: [
5     { type: "domicile", ville: "Paris" },
6     { type: "travail", ville: "Lyon" }
7   ]
8 }
```

8.1.2 Quand référencer

- Relation un-à-plusieurs illimitée
- Données rarement lues ensemble
- Besoin d'accès indépendant
- Éviter la duplication

8.2 Erreurs courantes à éviter

! Attention

1. Tableaux sans limite :

Ne JAMAIS créer de tableaux qui peuvent croître indéfiniment.

2. Skip pour pagination :

Sur de gros volumes, préférer la pagination par curseur.

3. Requêtes sans index :

Toujours créer des index sur les champs fréquemment interrogés.

8.3 Optimisation des requêtes

- Placez \$match au début des pipelines
- Utilisez les projections pour limiter les données
- Limitez les résultats avec limit()
- Préférez \$in à \$or pour le même champ
- Évitez \$where il ne peut pas utiliser d'index

9 Exemple pratique complet

9.1 Tableau de bord des stagiaires

```
1 db.Stagiaires.aggregate([
2   { $facet: {
3     "repartitionVilles": [
4       { $group: {
5         _id: "$ville",
6         count: { $sum: 1 }
7       }},
8       { $sort: { count: -1 } },
9       { $limit: 10 }
10    ],
11    "statistiquesAge": [
12      { $group: {
13        _id: null,
14        moyenne: { $avg: "$age" },
15        minimum: { $min: "$age" },
16        maximum: { $max: "$age" }
17      }}
18    ],
19    "topCompetences": [
20      { $unwind: "$competences" },
21      { $group: {
22        _id: "$competences",
23        count: { $sum: 1 }
24      }},
25      { $sort: { count: -1 } },
26      { $limit: 10 }
27    ]
28  }}
29 ])
```

10 Résumé des commandes essentielles

Opération	Commande	Description
Insertion	<code>insertOne()</code> , <code>insertMany()</code>	Créer des documents
Lecture	<code>find()</code> , <code>findOne()</code>	Rechercher des documents
Mise à jour	<code>updateOne()</code> , <code>updateMany()</code>	Modifier des documents
Suppression	<code>deleteOne()</code> , <code>deleteMany()</code>	Supprimer des documents
Agrégation	<code>aggregate()</code>	Pipeline de transformation
Index	<code>createIndex()</code> , <code>dropIndex()</code>	Gestion des index
Analyse	<code>explain()</code>	Analyser les performances

Table 9: Commandes MongoDB essentielles

10.1 Index recommandés pour la collection Stagiaires

```

1 // Recherche par nom
2 db.Stagiaires.createIndex({ nom: 1, prenom: 1 })
3
4 // Filtrage géographique
5 db.Stagiaires.createIndex({ ville: 1, age: -1 })
6
7 // Recherche par compétence
8 db.Stagiaires.createIndex({ competences: 1 })
9
10 // Tri chronologique
11 db.Stagiaires.createIndex({ dateInscription: -1 })
12
13 // Email unique
14 db.Stagiaires.createIndex(
15     { email: 1 },
16     { unique: true }
17 )

```

Conclusion

Ce cours couvre l'ensemble des fonctionnalités essentielles de MongoDB, des opérations CRUD de base jusqu'aux techniques avancées d'agrégation et d'optimisation. La pratique régulière avec des projets réels vous permettra de maîtriser ces concepts et d'optimiser vos applications.

Prochaines étapes :

- Pratiquez avec vos propres collections
- Explorez la documentation officielle MongoDB
- Apprenez la réplication et le sharding
- Étudiez les transactions multi-documents